

研究背景

随着云计算和外包存储的普及，用户需要将私密数据托管到不完全可信的服务器上，同时仍希望能够对数据进行高效的检索和更新。传统的对称可搜索加密（Searchable Symmetric Encryption, SSE）方案仅支持静态数据库，无法满足动态环境下的插入和删除需求。为此，动态 SSE（Dynamic SSE）应运而生，允许客户端在保持数据机密性的前提下，对远程托管的数据库进行插入（Add）与删除（Del）操作，同时支持关键字查询（Search）。

然而，随着对隐私泄露风险认识的加深，研究者发现仅满足“查询不泄露文档内容”的基本要求还远远不够。特别是：

- **前向隐私（Forward Privacy）**

当用户向数据库插入一个新的关键字-文档对 $(w, ind)(w, \mathit{ind})$ 时，服务器不应能将该插入与之前的任何搜索结果关联，以防止注入攻击（injection attacks）。

- **后向隐私（Backward Privacy）**

当用户删除一个已经存在的关键字-文档对 $(w, ind)(w, \mathit{ind})$ 后，后续对关键字 w 的查询不应泄露该已删除文档的信息，否则服务器可通过追踪删除前后的查询结果差异还原敏感记录。

在此前工作中，虽然已有多种动态 SSE 方案实现了高效的搜索与更新，但大多数在设计上只关注了查询时的最小泄露，或具备前向隐私，极少数考虑到后向隐私。即便是少数提及后向隐私的方案，也缺乏形式化定义与高效实现。

鉴于此，Bost 等人在“Forward and Backward Private Searchable Encryption from Constrained Cryptographic Primitives”（CCS 2017）一文中，首次针对后向隐私给出了**多种分级定义**，并提出了一系列既满足前向隐私又满足后向隐私的动态 SSE 构造，填补了该领域的空白。

本文指出的之前研究的不足

1. **缺乏形式化的后向隐私定义**

尽管 Stefanov et al. 2014（SPS14）最早提出“查询后不应泄露已删除条目”的概念，但并未给出完善的安全模型与证明，且相关后续工作多未深入讨论。

2. **现有方案仅专注前向隐私或需要 ORAM 才能实现后向隐私**

- 例如 Cash et al. 2014、Bost 2016 等方案关注前向隐私，但未考虑删除后信息的彻底隐藏。
- 若要获得后向隐私，通常借助昂贵的 ORAM (Oblivious RAM) 构造（如 TWORAM [GMP16]），导致巨大性能开销，难以实用。

3. **缺乏兼顾效率与隐私的可用构造**

已有方案若求最强后向隐私，往往要牺牲多轮交互或带来线性于数据库规模的复杂度，无法满足实际大规模部署需求。

本文的核心创新点

1. **提出并形式化了多层次的后向隐私定义**

非正式地说，如果每当将关键字/文档对 $(w, ind)(w, ind)$ 添加到数据库中**然后删除**时，对 w 的**后续搜索查询不会显示 ind** ，则SSE方案是向后私有的（或向后安全的）

- **插入模式泄露后向隐私（Insertion-Pattern Backward Privacy, Level I）**

仅泄露当前存在于数据库的匹配文档及它们的插入时间（不含删除信息），并泄露至今对关键字 w 的总更新次数。

- **更新模式泄露后向隐私 (Update-Pattern Backward Privacy, Level II)**
在上述基础上，还泄露所有针对关键字 w 的更新发生的具体时间戳（但不泄露是插入还是删除哪个文档）。
- **弱后向隐私 (Weak Backward Privacy, Level III)**
在更新模式泄露的基础上，还泄露“哪次删除对应哪次插入”（即 DelHist 信息）。
本文针对这三种不同强度的后向隐私，分别给出了清晰的形式化定义，对整个领域具有奠基意义。

2. 引入受限伪随机函数 (Constrained PRF) 和可打孔加密 (Puncturable Encryption) 构造高效实现

- **FS-RCPRF 框架**
首次将“范围受限 PRF (Range-Constrained PRF)”应用于动态 SSE，通过对 PRF 的“区间约束”来实现前向隐私。
- **Puncturable Encryption 框架**
借助 Green & Miers 2015 提出的“增量可打孔加密”，实现动态密钥更新，从而在查询时屏蔽已删除条目，满足弱后向隐私。
- 基于以上两个高级原语，文章设计了三个独立但相互补充的具体方案：
 1. **Moneta** (基于 TWORAM, $\Sigma\phi\phi\phi$ + FS-RCPRF 改造): 作为“基线”构造，用于将任意前向隐私 SSE 转化为后向隐私，但性能有所牺牲。
 2. **Diana / Diana_{del}** (仅用 GGM-基 PRF + FS-RCPRF/受限 PRF_{del}): 实现了极低开销的前向隐私 SSE (Diana)，以及在此基础上通过增量约束实现弱后向隐私 (Diana_{del})。
 3. **Janus** (基于 Puncturable Encryption): 在单轮交互下实现前向隐私且弱后向隐私，性能优异。

3. 系统化地比较了不同强度的后向隐私与效率折中

文章给出了所有方案与已有工作的详细对比（见 Table 1），包括计算、通信、客户端存储、轮次等指标，系统地展示了在不同隐私等级下的最优折中点。

Table 1 – Comparison with prior work. N is the number of keyword/document pairs in the database, K the number of distinct keywords, and D the number of documents. n_w is the size of the search result set for keyword w , a_w is the number of entries matching w inserted in total, while d_w is the number of deleted entries matching w (and $n_w = a_w - d_w$). The RT column stands for the number of roundtrips in the search protocol. FP (resp. BP) stands for forward (resp. backward) privacy. We denote different levels of backward privacy with I, II, and III, where I is the strongest level (see Section 4.2 for details). The notation $\tilde{O}$ hides polylog factors.

Scheme	Computation		Communication			Client Storage	FP	BP
	Search	Update	Search	RT	Update			
Π^{dyn} [CJJ ⁺ 14]	$O(a_w)$	$O(1)$	$O(n_w)$	1	$O(1)$	$O(1)$	✗	-
SPS [SPS14]	$O(\min\{a_w + \log N, n_w \log^3 N\})$	$O(\log^2 N)$	$O(n_w + \log N)$	1	$O(\log N)$	$O(N^\alpha)$	✓	-
TWORAM [GMP16]	$\tilde{O}(n_w \log N + \log^3 N)$	$\tilde{O}(\log^2 N)$	$\tilde{O}(n_w \log N + \log^3 N)$	2	$\tilde{O}(\log^3 N)$	$O(1)$	✓	-
$\Sigma\phi\phi\phi$ [Bos16]	$O(a_w)$	$O(1)$	$O(n_w)$	1	$O(1)$	$O(K \log D)$	✓	-
ARX [PBP16]	$O(a_w)$	$O(\log a_w)$	$O(n_w + \log a_w)$	1	$O(1)$	$O(K \log D)$	✓	-
Moneta § 4.3	$\tilde{O}(a_w \log N + \log^3 N)$	$\tilde{O}(\log^2 N)$	$\tilde{O}(a_w \log N + \log^3 N)$	3	$\tilde{O}(\log^3 N)$	$O(1)$	✓	I
Fides § 4.4	$O(a_w)$	$O(1)$	$O(a_w)$	2	$O(1)$	$O(K \log D)$	✓	II
Diana § 5.2	$O(a_w)$	$O(\log a_w)$	$O(n_w + \log a_w)$	1	$O(1)$	$O(K \log D)$	✓	-
Diana _{del} § 5.3	$O(a_w)$	$O(\log a_w)$	$O(n_w + d_w \log a_w)$	2	$O(1)$	$O(K \log D)$	✓	III
Janus § 6	$O(n_w \cdot d_w)$	$O(1)$	$O(n_w)$	1	$O(1)$	$O(K \log D)$	✓	III

本文提出的方案优于前人的方面

1. 更高效的前向隐私实现 – Diana

- **基于 GGM PRF 的 Range-Constrained PRF**
在插入时仅需要 $\mathcal{O}(\log n_w)$ 次 PRG 调用，其中 n_w 是关键字 w 的当前匹配总数；查询时服务器也只需 $\mathcal{O}(n_w)$ 次 PRG 调用。

- 与基于 RSA 的 $\Sigma\phi\phi\phi$ [Bos16] 相比，完全基于轻量对称密码运算，实验中对大规模数据集常见 $n_w \approx 10^4$ 的场景下，Diana 比 $\Sigma\phi\phi\phi$ 快 $5\times-10\times$ 。

2. 填补后向隐私空白，且性能可控

- **Moneta / Fides (基线方案)**

在已有任意前向隐私 SSE Σ 上做“包裹”改造，只额外引入不超过 $\mathcal{O}(a_w)$ 的线性通信开销（其中 a_w 为关键字 w 的加密记录总数）。

- **Diana_{del} (弱后向隐私)**

在保持 Diana 轻量特性的同时，通过在搜索时对已删除条目做“PRF 打孔”约束，实现弱后向隐私；与 Fides 等基线方案相比，节省大量存储与通信代价。

3. 单轮交互下的后向隐私 - Janus

- 在单轮查询中完成对已插入/已删除条目的区分：服务器仅能解密尚未删除的文档索引。
- 只需一次客户端 $\rightarrow$ 服务器查询，没有额外轮次；客户端与服务器之间传输量为 $\mathcal{O}(\log a_w + \log d_w)$ ，其中 a_w 和 d_w 分别为插入和删除记录数。
- 相比 TWORAM 等 ORAM 方案极大提升了性能，相对于只具备前向隐私的方案，仅在打孔密钥生成与传输上略增。

4. 首次将“受限 PRF”与“可打孔加密”原语完整引入 SSE，形成可重用的理论框架

- FS-RCPRF 框架可用来快速构造所有基于受限 PRF 的前向隐私 SSE，只要受限 PRF 效率够高，就能直接获得高效方案。
- Puncturable Encryption（尤其是其增量打孔性质）可在未来灵活扩展为支持更丰富查询类型或更强后向隐私的方案。
这为后续以更强可定制化 PRF/PE 原语为核心的安全查询方案设计提供了新思路。

核心方案

Moneta：基于 TWORAM 的 $B'(\Sigma)$ 方案

$B(\Sigma)$ ：简单的后向安全方案

Σ 作为任意的 SSE 方案。 $B(\Sigma)$ 方案包含三个算法：

1. Setup(DB):

直接运行底层 SSE 方案 Σ 的 Setup 算法，同时生成一个随机密钥 K_Σ 。

2. Update(K_Σ , add, w , ind, σ ; EDB)

客户端通过 PRF 用随机密钥 K_Σ 生成关键词密钥 K_w ，然后运行底层 SSE 方案 Σ 的 Update 算法。相比于直接上传文档索引 ind，客户端上传的是文档 id 及其操作符 op 用 K_w 加密后的密文 $E_{K_w}(\text{ind}, \text{op})$ 。

3. Search(K_Σ , w , σ ; EDB)

搜索算法返回一组匹配的加密文档索引 $E_{K_w}(\text{ind}, \text{op})$ ，客户端解密后剔除删除项得到最终结果。

Algorithm 1 Generic backward-private scheme $B(\Sigma)$ where Σ is an arbitrary SSE scheme and F is a PRF.

Setup(DB) :

1: $\Sigma.\text{Setup}(\text{DB}), K_\Sigma \xleftarrow{\$} \{0, 1\}^\lambda$

Search(K_Σ, w, σ ; EDB)

1: Client and Server run $\Sigma.\text{Search}(w)$, the client gets the list of results R .

Client:

2: $K_w \leftarrow F(K_\Sigma, w)$

3: Decrypt R as $(E_{K_w}(\text{ind}_1, \text{op}_1), \dots, E_{K_w}(\text{ind}_n, \text{op}_n))$

4: Return $\{\text{ind} : \exists i, (\text{ind}_i, \text{op}_i) = (\text{ind}, \text{add})$
 $\wedge \forall j > i, (\text{ind}_j, \text{op}_j) \neq (\text{ind}, \text{del})\}$

Update($K_\Sigma, \text{add}, w, \text{ind}, \sigma$; EDB)

1: Client: $K_w \leftarrow F(K_\Sigma, w)$

2: Client and Server run $\Sigma.\text{Update}(\text{add}, w, E_{K_w}(\text{ind}, \text{op}))$

$B(\Sigma)$ 作为一种简单的后向安全方案，依赖于底层的 SSE 方案能够实现二级或一级的后向安全。然而 $B(\Sigma)$ 也有一些缺点：

首先就是服务器不知道真正的文档信息，所以无法直接返回匹配的文档。不过，客户端可以将解密后的文档 id 再传给服务器，然后，服务器将相关的文档发送到客户端，这就需要多一轮通信，因此 $B(\Sigma)$ 是两轮往返的协议。

第二个缺陷是删除的文档在服务器中并没有真正被删除。不过只需要搜索后客户端重传搜索结果即可解决。

$B'(\Sigma)$: 对 $B(\Sigma)$ 的改进

通过对上述两个缺陷的改进，得到新的方案 $B'(\Sigma)$ 。

在这个方案中，客户端会对关键词 w 的查询次数进行跟踪，将其存在表 $\mathbf{T}$ 中。每次查询完，客户端更新表 $\mathbf{T}$ 中关键词 w 的值，使用从 w 和 $\mathbf{T}[w]$ 计算出的新密钥对结果进行重新加密，将其作为新的更新发送回服务器。为了迎合这项改动，数据库 DB 中的关键词替换为 关键词||其查询次数

Algorithm 2 Improved backward-private scheme $B'(\Sigma)$.

Setup(DB) :

- 1: $T[w] \leftarrow 0$ for all w , $K_\Sigma \xleftarrow{\$} \{0, 1\}^\lambda$
- 2: $DB' \leftarrow DB$ where keywords w are replaced by $w||0$
- 3: $\Sigma.\text{Setup}(DB')$

Search(K_Σ, w, σ ; EDB)

- 1: Client: $K_w \leftarrow H(K_\Sigma, w, T[w])$
- 2: Client and Server run $R \leftarrow \Sigma.\text{Search}(w||T[w])$ $\triangleright$ Server can erase all retrieved elements from memory
- Client:
- 3: Decrypt R as $(E_{K_w}(\text{ind}_1, \text{op}_1), \dots, E_{K_w}(\text{ind}_n, \text{op}_n))$
- 4: $R' \leftarrow \left\{ \text{ind} : \exists i, (\text{ind}_i, \text{op}_i) = (\text{ind}, \text{add}) \right. \\ \left. \wedge \forall j > i, (\text{ind}_j, \text{op}_j) \neq (\text{ind}, \text{del}) \right\}$
- 5: Send R' to Server
- 6: $T[w] \leftarrow T[w] + 1$
- 7: **for all** $\text{ind} \in R'$ **do** $\triangleright$ In parallel
- Run $\text{Update}(K_\Sigma, \text{add}, w, \text{ind}, \sigma; \text{EDB})$
- 9: **end for**

Update($K_\Sigma, \text{add}, w, \text{ind}, \sigma$; EDB)

- 1: Client: $K_w \leftarrow H(K_\Sigma, w, T[w])$
 - 2: Client and Server run $\Sigma.\text{Update}(\text{add}, w||T[w], E_{K_w}(\text{ind}, \text{op}))$
-

Moneta

方案 $B'(\Sigma)$ 显然是后向安全的，因为只有客户端删除已删除的索引后，服务器才能知道文档 id 的内容。此外，由于文档索引在每次搜索后都会重新加密，因此它实现了定义4.2意义上揭示后向隐私的更新模式的概念。显然， $B'(\Sigma)$ 对任意 SSE 方案都能实现二级后向安全。

$B'(\text{TWORAM})$ 实例化命名为 Moneta，其能够实现一级后向安全。

Fides：一种前向和后向隐私 SSE 的基准方案

使用 $\Sigma\phi\phi\sigma$ 作为方案 $B'(\Sigma)$ 的底层 SSE 方案，我们将这一实例化命名为 Fides。

性能分析

Fides 的搜索过程需要两轮通信，具有 $\mathcal{O}(a_w)$ 的计算和通信复杂度；其更新复杂度都是最优的，保持了恒定的通信和计算。

Diana：开销非常低的前向安全 SSE 方案

FS-RCPRF: 基于范围限制 PRF 的前向安全 SSE 方案

受限伪随机函数 $\tilde{F}$

$$\tilde{F} : \{0, 1\}^\lambda \times \{0, \dots, n_{max}\} \rightarrow \{0, 1\}^\lambda$$

其中，令 $\tilde{F}.\text{Constrain}(K_w, C_c)$ 输出只允许计算 $\tilde{F}$ 在 $\{0, \dots, c\}$ 上伪随机令牌的受限密钥 ST。而 $\tilde{F}.\text{Eval}(ST, i)$ 能够计算 $i \in \{0, c\}$ 中的伪随机令牌 T_i 。

该方案包含三个算法：

1. Setup():

客户端生成全局 SSE 密钥 K_Σ ，并初始化空映射 W ，用于记录每个 w 的当前计数。

服务器存储空的 EDB。

2. Update(K_Σ , add, w , ind, σ ; EDB)

- 客户端用 PRF F 对 w 产生两段子密钥 $K_w \parallel K'_w$ ，并取 $c \leftarrow W[w]$ (若 w 此前未出现，则 $c \leftarrow -1$)。计算

$$T_w^{c+1} \leftarrow \tilde{F}(K_w, c+1), UT_{c+1} \leftarrow H_1(K'_w, T_w^{c+1}), e \leftarrow \text{ind} \oplus H_2(K'_w, T_w^{c+1})$$

- 客户端将 $U \parallel e$ 上传给服务器，服务器仅将 (U, e) 存入 EDB (作为关键字 w 的倒排索引条目)。
- 客户端令 $W[w] \leftarrow c+1$

FS-RCPRF 的思想是，关键词 w 的更新令牌是在计数器模式下用 $\tilde{F}$ 生成的。每次插入新条目时，计数器都会自增。

注意到更新过程使用了两个不同的密钥 K_w 和 K'_w ，即使 K'_w 泄露，依赖 $\tilde{F}$ 的伪随机性，攻击者无法生成未来的更新令牌 UT_{c+1} 。

3. Search(K_Σ , w , σ ; EDB)

- 客户端计算两段子密钥 $K_w \parallel K'_w$ 。并取出 $c \leftarrow W[w]$ ，若 w 此前未出现，则返回 $\perp$ 。
- 客户端计算本次查询的搜索令牌 $ST = \tilde{F}.Constrain(K_w, C_c)$ ，然后与 K'_w ， c 组成受限密钥发送给服务器
- 服务器收到受限密钥后，遍历 $i = c, c-1, \dots, 0$ ，对每个 i 计算

$$T_i = \tilde{F}.Eval(ST, i), UT_i = H_1(K'_w, T_i)$$

若 $EDB[UT_i]$ 存在，则令

$$\text{ind} = e \oplus H_2(K'_w, T_i),$$

并将 ind 添加到返回列表，遍历完成后返回给客户端。

由于 i 仅遍历 $0 \leq i \leq c$ ，对应已有插入，任何 $i > c$ 的 T_i 无法预测，这就保证了前向隐私。

Algorithm 3 FS-RCPRF: Forward private SSE scheme from range-constrained PRF $\tilde{F}$. H_1 and H_2 are hash functions.

Setup()

- 1: $K_\Sigma \xleftarrow{\$} \{0, 1\}^\lambda$, $W, EDB \leftarrow \text{empty map}$
- 2: **return** (EDB, K_Σ , W)

Search(K_Σ , w , σ ; EDB)

Client:

- 1: $K_w \parallel K'_w \leftarrow F_{K_\Sigma}(w)$, $c \leftarrow W[w]$ $\triangleright c = n_w - 1$
- 2: **if** $c = \perp$ **then return** $\emptyset$
- 3: $ST \leftarrow \tilde{F}.Constrain(K_w, C_c)$ $\triangleright C_c$ is the circuit evaluating to 1 on $\{0, \dots, c\}$
- 4: Send (K'_w, ST, c) to the server.

Server:

- 5: **for** $i = c$ **to** 0 **do**

- 6: $T_i \leftarrow \tilde{F}(ST, i)$
- 7: $UT_i \leftarrow H_1(K'_w, T_i)$
- 8: $e \leftarrow EDB[UT_i]$
- 9: $\text{ind} \leftarrow e \oplus H_2(K'_w, T_i)$
- 10: Output each ind
- 11: **end for**

Update(K_Σ , add, w , ind, σ ; EDB)

Client:

- 1: $K_w \parallel K'_w \leftarrow F(K_\Sigma, w)$, $c \leftarrow W[w]$
- 2: **if** $c = \perp$ **then** $c \leftarrow -1$
- 3: $T_w^{c+1} \leftarrow \tilde{F}(K_w, c+1)$, $W[w] \leftarrow c+1$
- 4: $UT_{c+1} \leftarrow H_1(K'_w, T_w^{c+1})$, $e \leftarrow \text{ind} \oplus H_2(K'_w, T_w^{c+1})$
- 5: Send (UT_{c+1}, e) to the server.

Server:

- 6: $EDB[UT_{c+1}] \leftarrow e$

Diana, FS-RCPRF 的 GGM 实例

Diana 是 FS-RCPRF 的一个实例化方案，直接使用基于 GGM 的 BRC (Best Range Cover) 方法实现 $\tilde{F}$ 。它通过构造一棵完全二叉树实现：设 $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ 是伪随机生成器 (PRG)。每个非叶节点 k 经 $G(k)$ 扩展为 $G_0(k) \parallel G_1(k)$ ，最后的叶节点输出即 $F_{GGM}(K, x) = G_{x_{n-1}}(\dots(G_{x_1}(G_{x_0}(K))))$ ，其中 $x_{n-1} \dots x_0$ 是 x 的二进制表示。

要约束到范围 $[0, c]$ ，只需采集二叉树中覆盖 $[0, c]$ 所有叶子的“最小完全子树”节点作为约束密钥 ST (即“best range cover”方法)，节点数最多 $\mathcal{O}(\log n_{\max})$ 。

性能分析

更新时需要 $\mathcal{O}(\log n_{\max})$ 次 PRG 调用，用于从根节点计算新的叶子结点。

搜索时，通信复杂度为 $\mathcal{O}(\log n_w)$ (约束密钥有 $\mathcal{O}(\log n_w)$ 个结点)。服务器需枚举区间 $[0, c]$ 的覆盖节点：总共 $\mathcal{O}(n_w)$ 个节点，每个节点一次 PRG 调用。

Sophos 方案实际上使用了基于 TDP 的范围约束 PRF，其约束函数和计算函数分别为：

$$\begin{aligned}\tilde{F}.Constrain((SK, ST_0), n) &= ((PK), \pi_{SK}^{-n}(ST_0), n) = (PK, ST_n, n) \\ \tilde{F}.Eval((PK, ST_n, n), c) &= \pi_{SK}^{n-c}(ST_c)\end{aligned}$$

Diana_{del}：来自范围受限和可刺穿 PRF 的后向安全 SSE

值得注意的是，Diana 并不支持删除，然而可以通过让客户端和服务端维护两个实例，一个用于插入，一个用于删除来支持删除操作。

为了实现上述功能并具有后向安全，我们需要拓展之前的约束条件，使得底层约束 PRF $\tilde{F}$ 需同时满足两类约束：

1. **范围约束 (Range Constraint)**：限制计算范围为 $[0, c]$ (前向隐私)。
 2. **穿刺约束 (Puncturing Constraint)**：排除特定被删除条目 $x_1, x_2, \dots, x_n$ (后向隐私)。
- **约束谓词**： $C_{\{c, x_1, \dots, x_n\}}(x) = 1 \iff x \in [0, c] \text{ 且 } x \neq x_i (\forall i)$
服务器持有的约束密钥仅能计算满足上述条件的 PRF 值。

FS-RCPRF_{del}

FS-RCPRF_{del} 维护两个 SE 实例。当客户端需要插入计数器为 c 的关键词-文档对 (w, ind) 时，它按照 FS-RCPRF 的方式进行，并将该对插入到第一个 SE 实例中，但是客户端会额外生成一对 $(F'(K_w, (w, \text{ind})), \text{Enc}_{K'}(c))$ 发送给服务器存入一个映射表。当客户端要删除 (w, ind) 时，不直接告诉服务器下标 i ，而是往第二个 SE 实例 (删除实例) 里插入 $(w, F'(K_w, (w, \text{ind})))$ 。

查询时，客户端先向服务器发起对“删除实例”的搜索，服务器在删除实例中搜索关键字 w ，获取所有关联的标签 $F'(K_w, (w, \text{ind}))$ 和加密的计数器 $\text{Enc}_{K'}(c)$ 。然后解密 $\text{Enc}_{\{K'\}}(c)$ 得到被删除条目的位置 $x_1, x_2, \dots, x_n$ (即计数器值)，将其传回客户端。

客户端拿到删除下标列表 $\{x_1, \dots, x_n\}$ 之后，就可以基于这些值进行“打孔约束”，生成可以解出 $\{0, \dots, c\} \setminus \{x_1, \dots, x_n\}$ 上伪随机令牌的受限密钥 ST ；

最后，客户端用这个 ST 去对“插入实例”做一次标准的 FS-RCPRF 搜索，服务器遍历时会跳过那些被打孔的下标，从而不会泄露已删除的文档。

- 该设计保证了：
 - **前向隐私** (插入时生成的令牌依旧是随机的，不会与之前的搜索关联)；

- **弱后向隐私**（删除后，服务器只能看到哪些下标被打孔，却不知道具体哪个插入对应哪个删除发生，只知道“删除时刻”）；
- 但服务器仍能“知道何时发生了删操作”，所以只满足 Weak Backward Privacy (Level III)，而非更强的 Level I/II。
- **不足之处**
 - **多一轮额外通信**
客户端每次搜索首先要从“删除实例”取回删除下标的加密值，服务器再把它们解密后返回，才允许客户端生成最终的受限密钥并进行插入实例的搜索。相比单轮查询，存在额外的往返和较大数据量（删除下标列表往往较长）。
 - **只能实现弱后向隐私**
服务器在每次删除时，就会收到那条“删除令牌 τ ”，意味着它确切知道“哪一次查询之后发生了删除”且“删除了哪个下标对应的 τ ”——因此无法满足更高等级的后向隐私定义。

Diana_{del}

- FS-RCPRF_{del} 的 GGM PRF 实例化
- 当用 GGM PRF 来实现 $\tilde{F}$ 时，生成的受限密钥由“覆盖若干区间的树节点”组成：
 - 如果没有删除，直接覆盖 $[0, c]$ ；
 - 当有删除下标 $\{x_1, \dots, x_n\}$ ，令这些 x_i 按升序排列，则一共需要覆盖的区间是

$$[0, x_1 - 1], [x_1 + 1, x_2 - 1], \dots, [x_n + 1, c]$$
 - 对应 GGM 二叉树上要发送的节点数约为 $\mathcal{O}(d_w \cdot \log(n_w/d_w))$ ，其中 d_w 是删除次数， n_w 是插入次数。

因此，当删除次数较多时，打孔后的“约束密钥”会变得相当大，导致通信与服务器解算成本也会上升。

Janus：基于可刺穿加密的弱后向安全

Diana_{del} 在实现弱后向隐私时仍需在查询前上传删除集 D_w （通信依赖 d_w ），且分两步（先预取 ST 再检索索引）。为了进一步减少交互轮次与通信负载，Janus 利用了“增量可打孔加密”实现了**单轮**查询即可完成前向与弱后向隐私：

可打孔加密

与普通公钥加密不同，PE 允许在“打孔操作”后，密钥不再能解密绑定在特定标签 t 下的所有密文。

可刺穿加密（PE）方案是一种公钥加密方案，允许刺穿密钥以防止某些消息的解密。更确切地说，对于这样的方案，明文被加密并附加到标签上，密钥在一组标签上被刺破，从而无法解密附加到这些标签上的密文。

增量打孔

- PE 的秘密密钥可记为 $SK_n = (sk_0, sk_1, \dots, sk_n)$ ，打孔操作 $Puncture(SK_n, t)$ 会产出 $SK_{n+1} = (sk'_0, sk_1, \dots, sk_n, sk_{n+1})$ ，其中只需更新 sk_0 与新增一个小分量 sk_{n+1} ；
- 因此，“打孔到标签 t ”时，客户端仅需本地保存“最新打孔后的小分量 sk_0 ”，而“中间打孔产生的 $sk_1, \dots, sk_n$ ”可托管服务器。每次打孔仅在 sk_0 上本地少量计算，便产生可传输的 sk_{n+1} 。

被刺破的密钥（通常）会随着刺破次数的增加而增长（或非常大），将其存储在客户端是不切实际的。为了避免这个问题，我们使用了 Green Miers 方案的一个附加功能，我们称之为增量穿刺。

Janus 构造

Janus 使用两个前向安全的可搜索加密实例：

- Σ_{add} 用于存储用可刺破加密方案加密的新插入索引（插入实例）
- Σ_{del} 用于存储刺破的密钥元素（删除实例）。

所有与关键字 w 有关的“PE 参数”都由客户端本地的 $\text{PSK}[w]$ 管理——其中只保存 $sk_0[w]$ ，其余 sk_i 由服务器维护。具体算法如下：

1. Setup():

初始化 EDB_{add} 、 EDB_{del} 和 $\text{EDB}_{\text{cache}}$ ，以及空映射 PSK 、 SC 用于存储每个关键词的初始密钥和打孔计数。

生成全局密钥 $K_{\text{add}}, K_{\text{del}}, K_{\text{tag}}, K_S$ 。

2. Update($K_{\Sigma}, \text{op}, w, \text{ind}, \sigma; \text{EDB}$)

客户端计算标签 $t = F_{K_{\text{tag}}}(w, \text{ind})$ ，并取出 $sk_0 \leftarrow \text{PSK}[w]$ ， $i \leftarrow \text{SC}[w]$ ，若 $\text{PSK}[w] = \perp$ （即 w 首次插入），则重新生成 $\text{PSK}[w] \leftarrow \text{PPKE.KeyGen}()$ ， $\text{SC}[w] \leftarrow i$ 。

- 如果 $\text{op} = \text{add}$ （添加操作），则用 $sk_0 \leftarrow \text{PSK}[w]$ 对 ind 和 t 进行 PE 加密 $\text{ct} \leftarrow \text{PPKE.Encrypt}(sk_0, \text{ind}, t)$ ，然后向 Σ_{add} 插入记录 $(w || i, (\text{ct}, t))$ 。注意，这里不变动 $\text{PSK}[w]$ 与 $\text{SC}[w]$ 。
- 如果 $\text{op} = \text{del}$ （删除操作），则对标签 t 进行 PE 刺穿 $(sk'_0, sk_t) \leftarrow \text{PPKE.IncPuncture}(sk_0, t)$ ，产生新的局部密钥 sk'_0 与用于服务器的“打孔输出密钥分量” sk_t ，然后向 Σ_{del} 插入记录 $(w || i, (sk_t, t))$ ，将 sk'_0 更新回 $\text{PSK}[w]$ 。

3. Search($K_{\Sigma}, w, \sigma; \text{EDB}$)（单轮完成以下两步交互，客户端 $\rightarrow$ 服务器）

- 客户端取 $i \leftarrow \text{SC}[w]$ （若 $i = \perp$ ，说明没有该关键字，返回 $\emptyset$ ），向服务器发送 $sk_0 \leftarrow \text{PSK}[w]$ 和标签种子 $\text{tkn} \leftarrow F(K_S, w)$ 。
- 客户端更新 $\text{PSK}[w] \leftarrow \text{PPKE.KeyGen}()$ ，（为下次插入生成全新未打孔密钥）；并令 $\text{SC}[w] \leftarrow i + 1$ ——这样保证**搜索完成后旧密钥不再可用**，避免重放解密已删除项；
- 客户端与服务器并行发起：
 - $\Sigma_{\text{add}}.\text{Search}(K_{\text{add}}, w || i, \sigma; \text{EDB})$ ：服务器获得所有 PE 密文 ct_i 及其标签 t_i^{add} ；
 - $\Sigma_{\text{del}}.\text{Search}(K_{\text{del}}, w || i, \sigma; \text{EDB})$ ：服务器获得所有已打孔的密钥分量 sk_j 及其标签 t_j^{del} ；
- 服务器用 $\text{SK} = (sk_0, sk_1, \dots, sk_m)$ （拼装成可解密的“最终 PE 密钥”）来尝试解密 Σ_{add} 返回的 (ct_i, t_i^{add}) ；只有 t_i^{add} 不在**打孔标签集中**时，才会成功输出 $(\text{ind}_i, t_i^{\text{add}})$ ；最后服务器得到所有新未删除的 (ind, t) ；
- 服务器从 $\text{EDB}_{\text{cache}}[\text{tkn}]$ 读取上次搜索缓存 OldInd （若无，则为空），从中剔除所有 t_j^{add} 在打孔标签集中的旧索引；再与当次解密“NewInd”合并成 Res ；更新缓存 $\text{EDB}_{\text{cache}}[\text{tkn}] \leftarrow \text{Res}$ ；将 Res （即所有 w 当前未删除文档索引附带标签）直接返回给客户端。

Algorithm 4 Janus: weakly backward-secure SSE.**Setup()**

```

1:  $(\text{EDB}_{\text{add}}, K_{\text{add}}, \sigma_{\text{add}}) \leftarrow \Sigma_{\text{add}}.\text{Setup}()$ 
2:  $(\text{EDB}_{\text{del}}, K_{\text{del}}, \sigma_{\text{del}}) \leftarrow \Sigma_{\text{del}}.\text{Setup}()$ 
3:  $K_{\text{tag}}, K_S \leftarrow \{0, 1\}^\lambda, \mathbf{PSK}, \mathbf{SC}, \text{EDB}_{\text{cache}} \leftarrow \text{empty map}$ 
4: return  $((\text{EDB}_{\text{add}}, \text{EDB}_{\text{del}}, \text{EDB}_{\text{cache}}),$ 
    $(K_{\text{add}}, K_{\text{del}}, K_{\text{tag}}, K_S), (\sigma_{\text{add}}, \sigma_{\text{del}}, \mathbf{PSK}, \mathbf{SC}))$ 

```

Search $(K_\Sigma, w, \sigma; \text{EDB})$ *Client:*

```

1:  $i \leftarrow \mathbf{SC}[w]$ .
2: if  $i = \perp$  return  $\emptyset$ 
3: Send  $sk_0 = \mathbf{PSK}[w]$  to the server.
4:  $\mathbf{PSK}[w] \leftarrow \text{PPKE.KeyGen}(1^\lambda), \mathbf{SC}[w] \leftarrow i + 1$ .
5: Send  $\text{tkn} \leftarrow F(K_S, w)$  to the server.
Client (C) & Server (S):
6: C and S run  $\Sigma_{\text{add}}.\text{Search}(K_{\text{add}}, w || i, \sigma_{\text{add}}; \text{EDB}_{\text{add}})$ . The server
   gets a list  $((ct_1, t_1^{\text{add}}), \dots, (ct_n, t_n^{\text{add}}))$  of ciphertexts and tags.
7: C and S run  $\Sigma_{\text{del}}.\text{Search}(K_{\text{del}}, w || i, \sigma_{\text{del}}; \text{EDB}_{\text{del}})$ . The server
   gets a list  $((sk_1, t_1^{\text{del}}), \dots, (sk_m, t_m^{\text{del}}))$  of key elements.
8: S decrypts the ciphertexts with  $\text{SK} = (sk_0, sk_1, \dots, sk_m)$ , and
   obtains the list  $\text{NewInd} = ((\text{ind}_1, t_1), \dots, (\text{ind}_\ell, t_\ell))$ .

```

Server:

```

9:  $\text{OldInd} \leftarrow \text{EDB}_{\text{cache}}[\text{tkn}]$ 
10: Remove from  $\text{OldInd}$  the indices whose tags are in  $\{t_j^{\text{del}}\}$ .
11:  $\text{Res} \leftarrow \text{OldInd} \cup \text{NewInd}, \text{EDB}_{\text{cache}}[\text{tkn}] \leftarrow \text{Res}$ 
12: return  $\text{Res}$ 
Update $(K_\Sigma, \text{add}, w, \text{ind}, \sigma; \text{EDB})$ 
1:  $t \leftarrow F_{K_{\text{tag}}}(w, \text{ind})$ 
2:  $sk_0 \leftarrow \mathbf{PSK}[w], i \leftarrow \mathbf{SC}[w]$ 
3: if  $sk_0 = \perp$  then
4:    $sk_0 \leftarrow \text{PPKE.KeyGen}(1^\lambda), \mathbf{PSK}[w] \leftarrow sk_0$ 
5:    $i \leftarrow 0, \mathbf{SC}[w] \leftarrow i$ 
6: end if
7: if  $\text{op} = \text{add}$  then
8:    $ct \leftarrow \text{PPKE.Encrypt}(sk_0, \text{ind}, t)$ 
9:   Run  $\Sigma_{\text{add}}.\text{Update}(K_{\text{add}}, \text{add}, w || i, (ct, t), \sigma_{\text{add}}; \text{EDB}_{\text{add}})$ 
10: else  $\triangleright \text{op} = \text{del}$ 
11:    $(sk'_0, sk_t) \leftarrow \text{PPKE.IncPuncture}(sk_0, t)$ 
12:   Run  $\Sigma_{\text{del}}.\text{Update}(K_{\text{del}}, \text{add}, w || i, (sk_t, t), \sigma_{\text{del}}; \text{EDB}_{\text{del}})$ 
13:    $\mathbf{PSK}[w] \leftarrow sk'_0$ 
14: end if

```

要点解析

- **前向隐私**：每次 Search 前，客户端都用旧 $\mathbf{PSK}[w]$ 提供给服务器一个“仅包含未打孔分量”的 PE 密钥；服务器仅能解密查询阶段之前插入且尚未删除的文档。对于**后续插入或删除**所对应的新“PE 密钥分量”，服务器都无法提前预测或关联。
- **弱后向隐私 (Level III)**：
 - 服务器在当前 Search 中仅能看到“在上次 Search 之后插入且未打孔”的 (ct, t) 。已打孔标签 t 对应的 ind 从未出现在“可解密”集合里，且旧查询缓存中相应旧索引也在“已打孔标签”检验时被剔除。
 - 服务器通过 Σ_{del} 返回的所有 (t_j^{del}) 即可知道“哪些插入曾经被删除”及“删除发生的时间戳”，因此只能达到弱后向隐私 (Level III)，无法满足更严格的 Level I/II。
- **单轮交互**：客户端只需一次下发 (sk_0, tkn) ；服务器在本轮内部并行执行 $\Sigma_{\text{add}}.\text{Search}$ 与 $\Sigma_{\text{del}}.\text{Search}$ ，它们都是单轮执行，因此 Janus 整体查询依旧是单轮。
- **缓存机制**：由于服务器从上次缓存 OldInd 可继续保留未删除索引，提高性能，同时不会“泄露”对数据库结构的额外信息。

安全性简析• **前向隐私**

来自 Σ_{add} 、 Σ_{del} 本身的前向隐私保证，以及 PE “已打孔分量”不会泄露将来要打孔的标签。服务器无法将当前插入与之前任何查询对应上。

• **弱后向隐私 (Level III)**

- **TimeDB(w)** 只泄露当前数据库中未删除索引集合 $\{\text{ind}_j\}$ 及其插入时刻；已删除索引不在服务端可解密集合中。
- **DelHist(w)** 泄露每个删除时刻以及被删除条目原来插入的 i （因为 Σ_{del} 在执行 $(w || i)$ 搜索时会返回所有历史打孔标签 t_{del} ），正好满足 Level III 定义。
- **不泄露更丰富信息**
服务器看不到任何 (i, t_i^{add}) 在被删除前后对应的文档内容，只能知道“标签 t ”曾被打孔，但无法还原文件具体内容。

主要结果与结论

1. 安全性方面

- **形式化定义**：首次给出三种后向隐私 (Level I/II/III) 的明确定义及其安全模型。
- **构造丰富**：提供了一套从任意前向隐私 SSE Σ 通过“包裹”改造得到 Level II/III 后向隐私的通用方法 (Moneta、Fides)，以及基于受限 PRF 的轻量前向隐私构造 (Diana) 与其 Level III 扩展 ($Diana_{del}$)，再以及基于增量可打孔加密的单轮 Level III 方案 (Janus)。
- **安全证明**：所有方案均在多轮交互适应性安全 (Adaptive Security) 下给出严格证明，展示了“受限 PRF + PE”原语在 SSE 领域的可行且安全的应用。

2. 性能评估 (见原文 Section 7 及相关图表；以下皆根据实验结果总结)

- **Diana vs $\Sigma\phi\phi\phi$**
 - 在百万级别文档与数十万关键字场景下，Diana 在插入与搜索两项操作中，比 Sophos (基于 RSA “跳表”式实现) 平均快 $5\times-10\times$ ，大文件集时峰值能接近 $15\times$ ；其通信与客户端负载也更低。
- **** $Diana_{del}$ vs Fides/Moneta ****
 - 在同样测试集合中， $Diana_{del}$ 的查询对 $n_w = 10^4$ 、 $d_w = 10^3$ 规模能在 0.2 秒以内完成，而 Fides 与 Moneta 动辄需要数秒级别；通信量也比 Fides 少 $\approx 50\%$ 。
- **Janus 性能**
 - 单轮交互带来的延迟非常低 (取决于 Σ_{add} 与 Σ_{del} 各自插入和搜索延迟之和)，在千万级数据集场景下，查询平均时延 ≈ 0.25 秒，且通信仅数十 KB，适合真正在线部署。
- **客户端存储**
 - Diana 仅需为每个 w 存储常数大小 (c_w 及 K_w, K'_w)；
 - $Diana_{del}$ 需额外存储 d_w 个“已删除位置信息”以构造 ST；
 - Janus 仅需保留每个 w 最新 sk_0 与计数 $SC[w]$ ，大幅减小客户端负担。

3. 结论与启示

- 本文首次从理论与实践上完整阐释了“后向隐私”在动态 SSE 中的多种层次，为今后更强隐私需求的加密搜索奠定了基石。
- **受限 PRF 与 可打孔加密** 成为 SSE 领域的重要构造模块：前者带来高效前向隐私，后者则为删除后的信息屏蔽提供安全保障。
- 文中所提出的诸多技术手段均可在未来向更丰富查询类型 (多关键字、多属性、范围查询等) 或更强隐私级别 (接近 Level I/II) 扩展。
- 通过与其他高级原语 (如功能性加密、同态加密等) 结合，有望进一步压缩通信与计算开销，实现功能更强但仍可用的安全可搜索加密服务。